SEGURIDAD INFORMÁTICA · UNIVERSIDAD NACIONAL

Laboratorio 6
Monitoreo de UNA Chat

Vercel, Cronitor, Sentry, OpenTelemetry, Grafana Cloud, Docker y Artillery

Estudiante: Gerald

Fecha: 31 de octubre de 2026

Repositorio:
github.com/Gerald120904/infosec-unachat-2026

Estado: despliegue público verificado; las tres señales OTLP locales se consultaron en Grafana Cloud.

01 · OBJETIVO

Objetivo y alcance

Instrumentar y verificar UNA Chat en su despliegue público y en un entorno local contenerizado. La práctica cubre disponibilidad, captura de errores, telemetría distribuida y carga funcional a través del protocolo real del chat.

Resultados objetivo

- Publicar el fork en Vercel con Node.js 24 y verificar el intercambio de mensajes Socket.IO sobre WebSocket.
- Monitorear el endpoint público con Cronitor y habilitar alertas.
- Preparar Sentry para registrar errores del entorno Vercel.
- Enviar logs, métricas y trazas únicamente desde la ejecución local/Docker hacia Grafana Cloud.
- Comparar 300 mensajes Socket.IO y 300 solicitudes HTTP de prueba exclusivamente contra localhost.

> **Regla de alcance:** Artillery nunca se ejecutó contra Vercel. No se enviaron logs locales, métricas ni trazas desde el runtime público Vercel hacia Grafana Cloud.

Estado comprobable

Componente	Estado
Vercel + Socket.IO	Deployment Ready; HTTP 200 y respuesta Socket.IO verificados.
Cronitor	URL asignada; Healthy, alertas activas.
Sentry	Error controlado recibido en Production; variables temporales retiradas.
Grafana Cloud	Loki, Prometheus y Tempo recibieron datos locales consultables.
Docker + Artillery	Operativo; carga local completada.

02 · ARQUITECTURA

Aplicación e instrumentación

UNA Chat utiliza Express para servir la interfaz y Socket.IO para recibir el evento `Evento-Mensaje-Server` y difundir el mensaje a los clientes conectados.

Cambios implementados

- Socket.IO usa la ruta `/api/socket-io/socket.io` en cliente, servidor y pruebas, con transporte WebSocket y reconexión.
- OpenTelemetry se inicializa antes de Express/HTTP; se agregaron spans de mensajes y métricas HTTP/chat.
- Pino produce eventos estructurados con `trace_id` y `span_id`, sin registrar el texto del chat.
- Sentry Node SDK solo inicia en Vercel y requiere `SENTRY_DSN`; se desactiva el envío de PII y se filtran cookies, cabeceras y query strings.
- La ruta HTTP de laboratorio `/__lab/message` solo está disponible fuera de Vercel y reutiliza el validador real.
- Docker publica el servicio solamente en `127.0.0.1:3000`.

> **Privacidad de telemetría:** la configuración impide exportar desde Vercel. Solo la ejecución local/Docker lee las credenciales OTLP desde el archivo ignorado `.env`.

Protocolo Socket.IO verificado

El cliente envía una cadena JSON con `nombre`, `mensaje` y `color` a `Evento-Mensaje-Server`. El servidor valida el contenido y lo difunde a los clientes mediante el evento de respuesta del chat. Las pruebas validan recepción en ambos clientes.

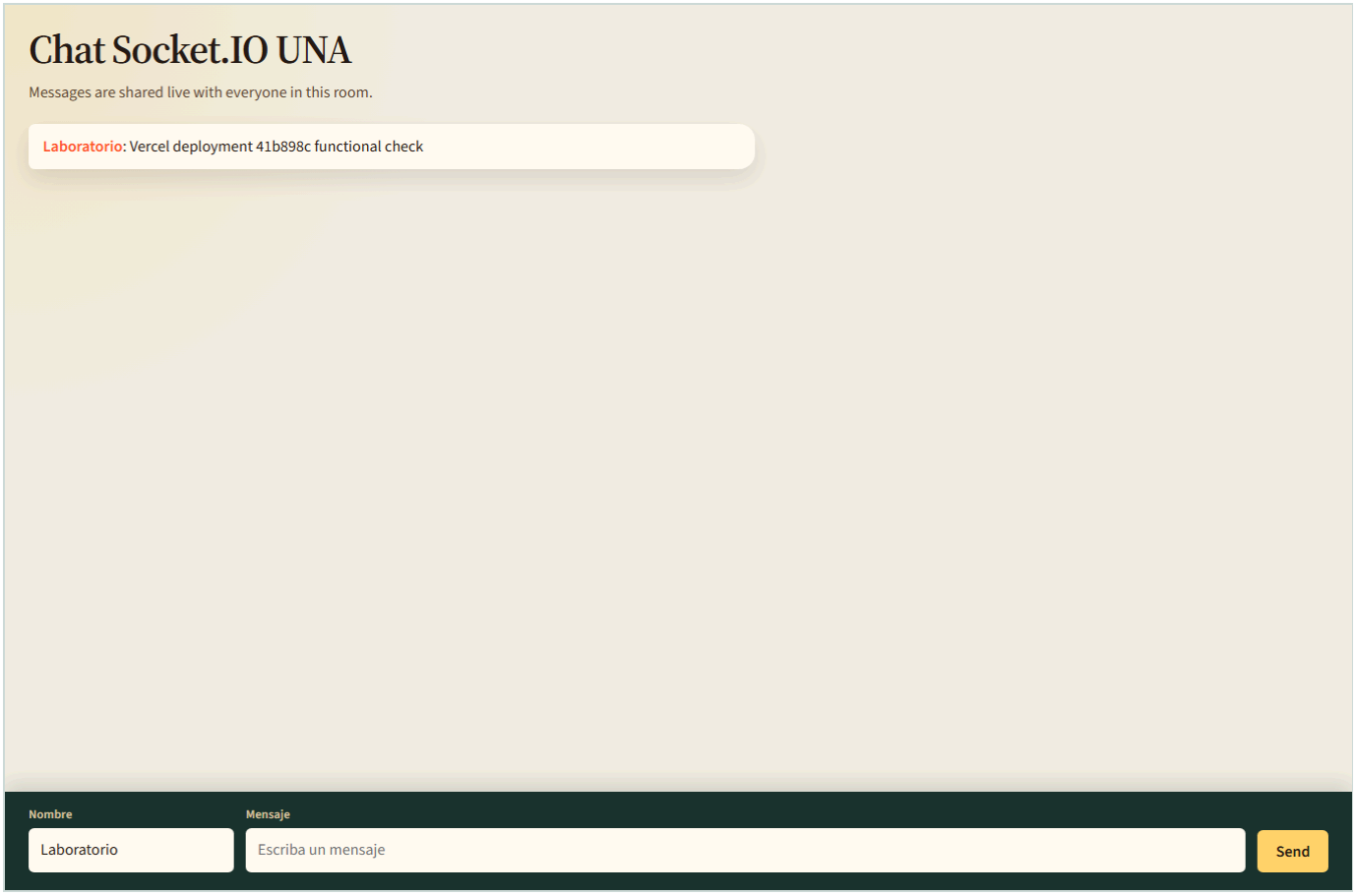
03 · VERCEL

Despliegue público

El proyecto Vercel está conectado al fork correcto y a la rama `master`. El panel mostró el deployment en estado Ready y la aplicación pública respondió correctamente.

URL pública asignada: https://infosec-unachat-2026-black.vercel.app

Repositorio: Gerald120904/infosec-unachat-2026

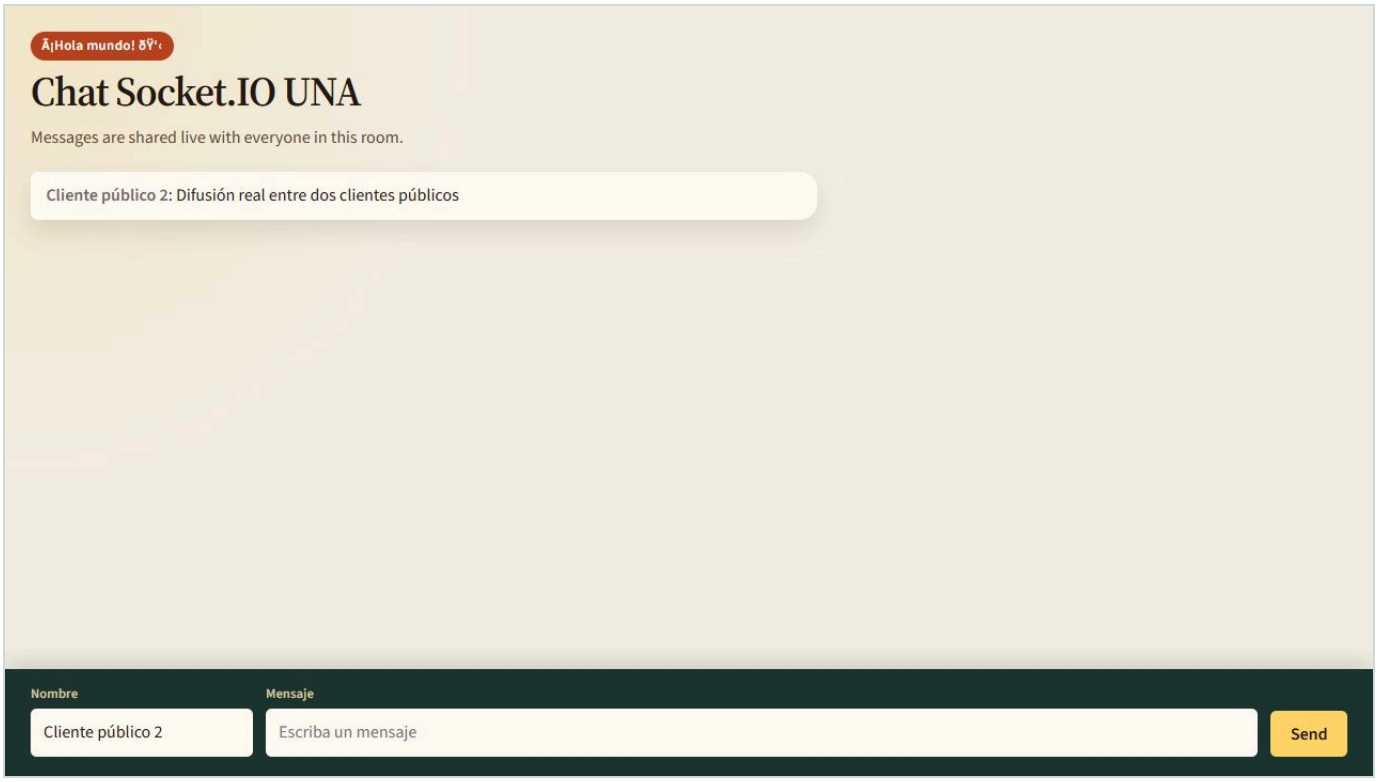


Captura Playwright original de la URL asignada tras el despliegue 0f998db. La página respondió HTTP 200, conectó por Socket.IO y muestra el mensaje de verificación; sin carga contra producción.

04 · SOCKET.IO

Conversación entre dos clientes

Se usaron dos páginas independientes del alias público. Ambas establecieron el transporte WebSocket y recibieron los mensajes publicados en la sala.



Comprobación funcional

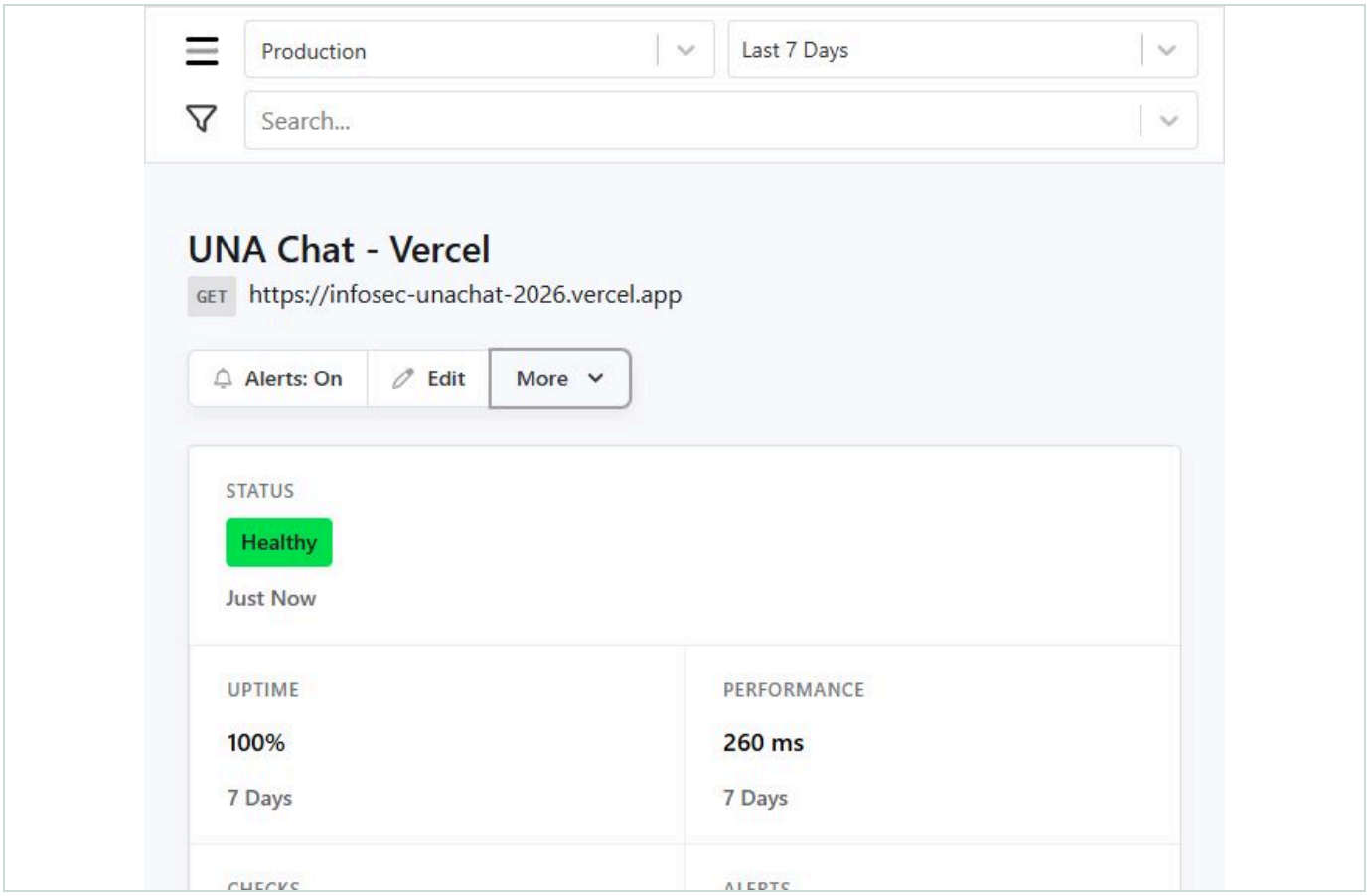
- Cliente 1 envió “Socket.IO en alias público de Vercel”.
- Cliente 2 envió “Difusión real entre dos clientes públicos”.
- La interfaz mostró los mensajes; se verificó el transporte WebSocket en ambos clientes.

Registro: `parte-1-vercel-sentry-cronitor/vercel-chat-test-2026-10-30.txt`

05 · CRONITOR

Disponibilidad y notificaciones

Se creó el monitor **UNA Chat - Vercel** para el alias público. La configuración es GET cada cinco minutos, timeout de 10 segundos, cuatro regiones y lista **Default**. Las alertas están habilitadas. El 31 de octubre se actualizó el endpoint a la URL asignada de producción.



Resultado observado

Primer chequeo real: Healthy HTTP 200, 260 ms desde Frankfurt. Uptime inicial mostrado: 100% para el período registrado.

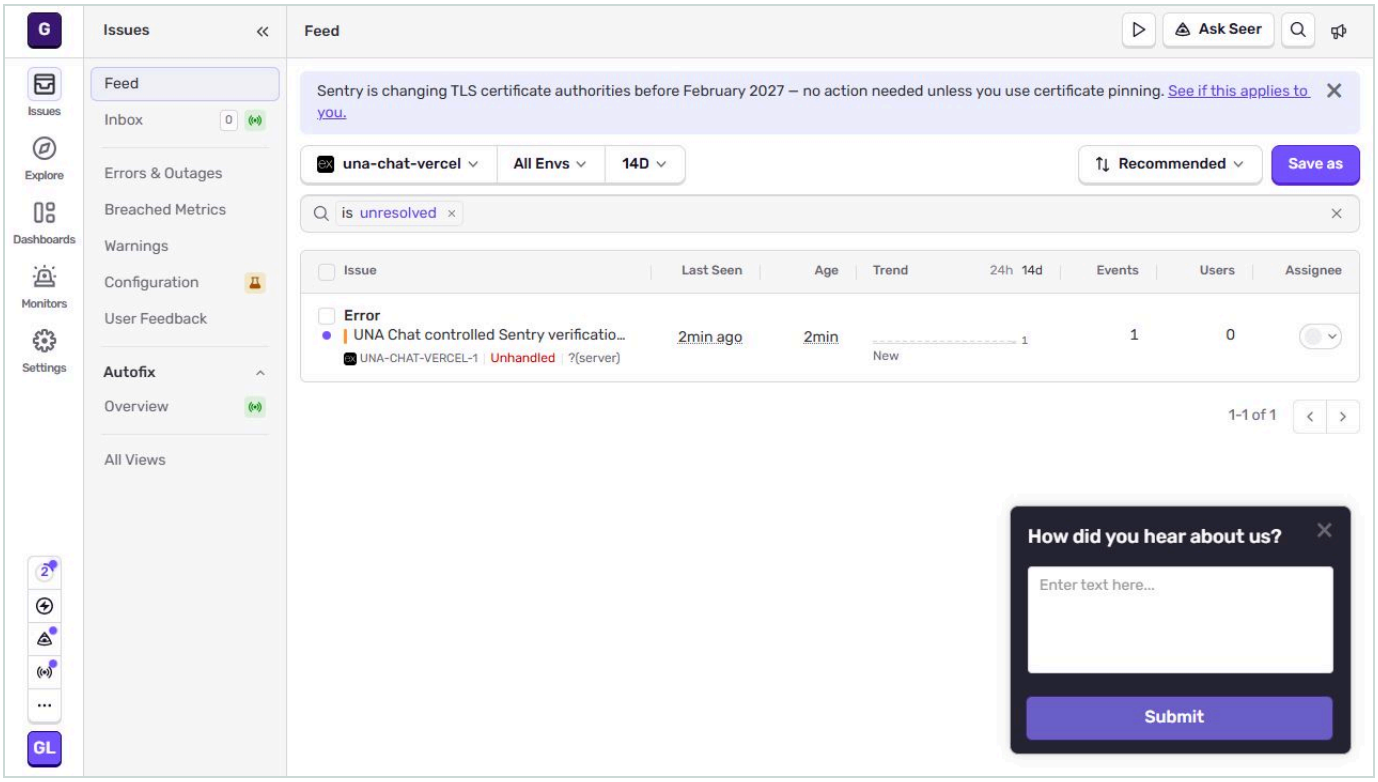
Monitor: cronitor.io/app/monitors/UTzIWf. El monitor anterior de Render se conservó separado.

La captura es del estado saludable inicial: 200 OK, 260 ms, alertas activas. El 31 de octubre el endpoint se actualizó a la URL asignada `infosec-unachat-2026-black.vercel.app`. La vista vigente mostró Healthy, 100% de uptime en 24 horas, promedio 341 ms, 19 chequeos y 0 alertas. Estado actual en `10-cronitor-vercel-final-2026-10-31.txt`; no se generó una caída artificial.

06 · SENTRY

Proyecto de captura de errores

En la organización `gerald-aw0` se creó el proyecto Express **una-chat-vercel**. El SDK Node se inicializa en Vercel; la captura de PII está desactivada. `SENTRY_DSN` es la única variable de aplicación que permanece en Production.



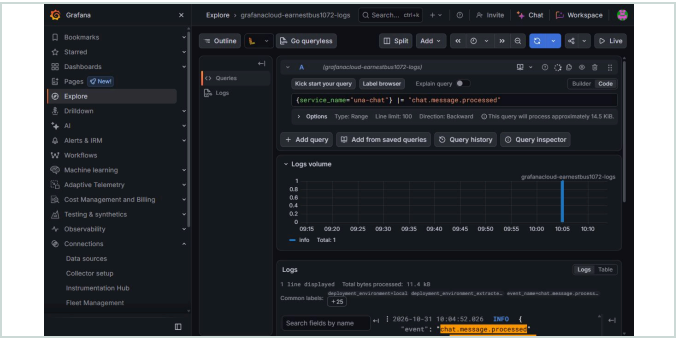
> **Resultado real:** Sentry recibió desde Vercel Production el error de verificación; el Feed mostró un evento con conteo 1. La captura original se tomó el 31 de octubre de 2026.

La ruta temporal y las variables `SENTRY_LAB_TEST_ENABLED` y `SENTRY_LAB_TEST_SECRET` se retiraron. El deployment de producción quedó Ready y la ruta ya responde 404. La evidencia no incluye DSN, claves ni datos de usuario.

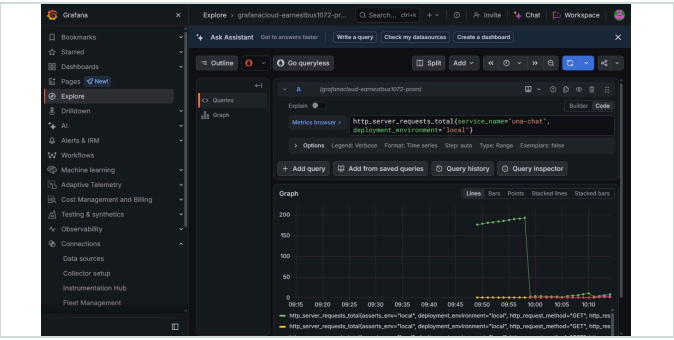
07 · GRAFANA CLOUD

Logs, métricas y trazas recibidos

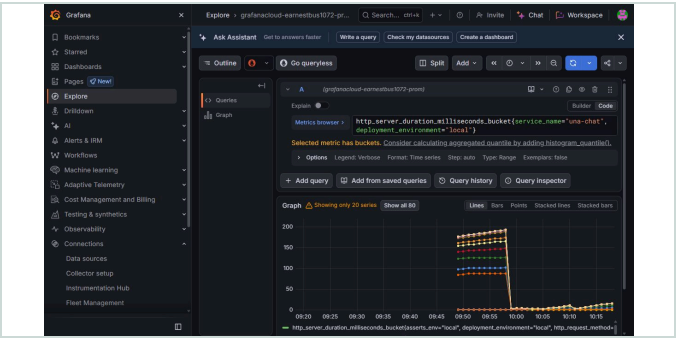
El endpoint del stack en `us-east-3` coincide con `.env.example`. La autenticación usa `Authorization=Basic%20…`; el parser OTLP decodifica `%20` a espacio. El token permanece en el `.env` local ignorado por Git y Vercel tiene exportación desactivada.



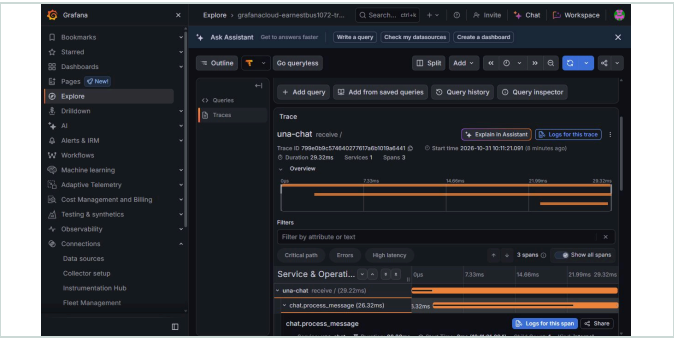
Loki · evento del chat · service=una-chat, environment=local · Last 1 hour.



Prometheus/Mimir · solicitudes HTTP y series del servicio local.



Prometheus/Mimir · buckets de latencia HTTP.



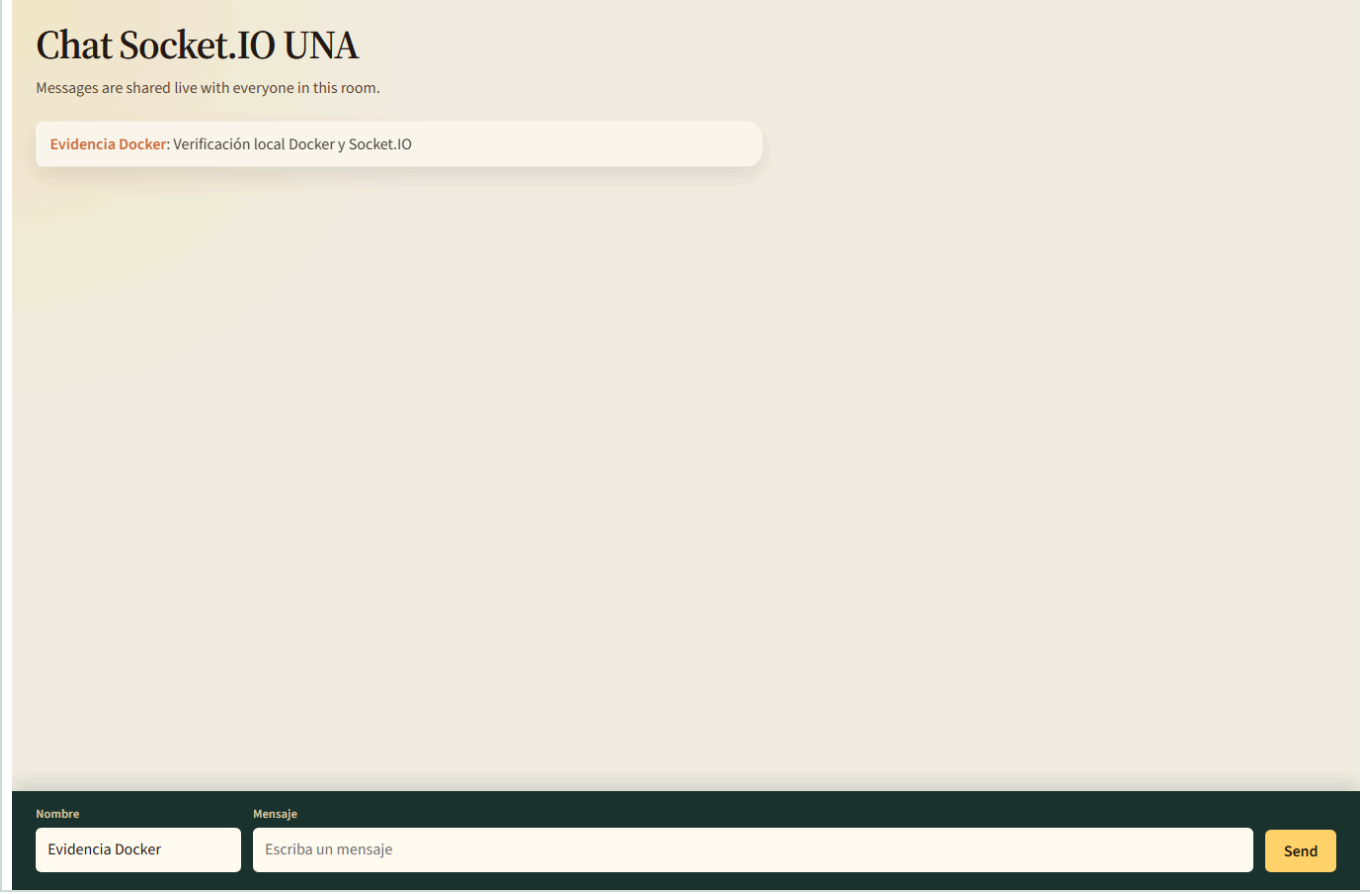
Tempo · traza de mensaje Socket.IO, tres spans, runtime Docker.

> **Correlación comprobada:** desde Tempo se abrió la consulta de logs de la traza y Loki encontró el evento con el mismo `trace_id`. La advertencia de `host.id` no impidió exportar. Las imágenes conservan el intervalo de consulta mostrado por Grafana.

08 · DOCKER

UNA Chat en contenedor local

Docker Engine 29.9.2 respondió a `docker info` y `docker version`. El servicio de Compose quedó Healthy en `127.0.0.1:3000`, sin detener ni modificar contenedores de otros proyectos.



Pruebas locales

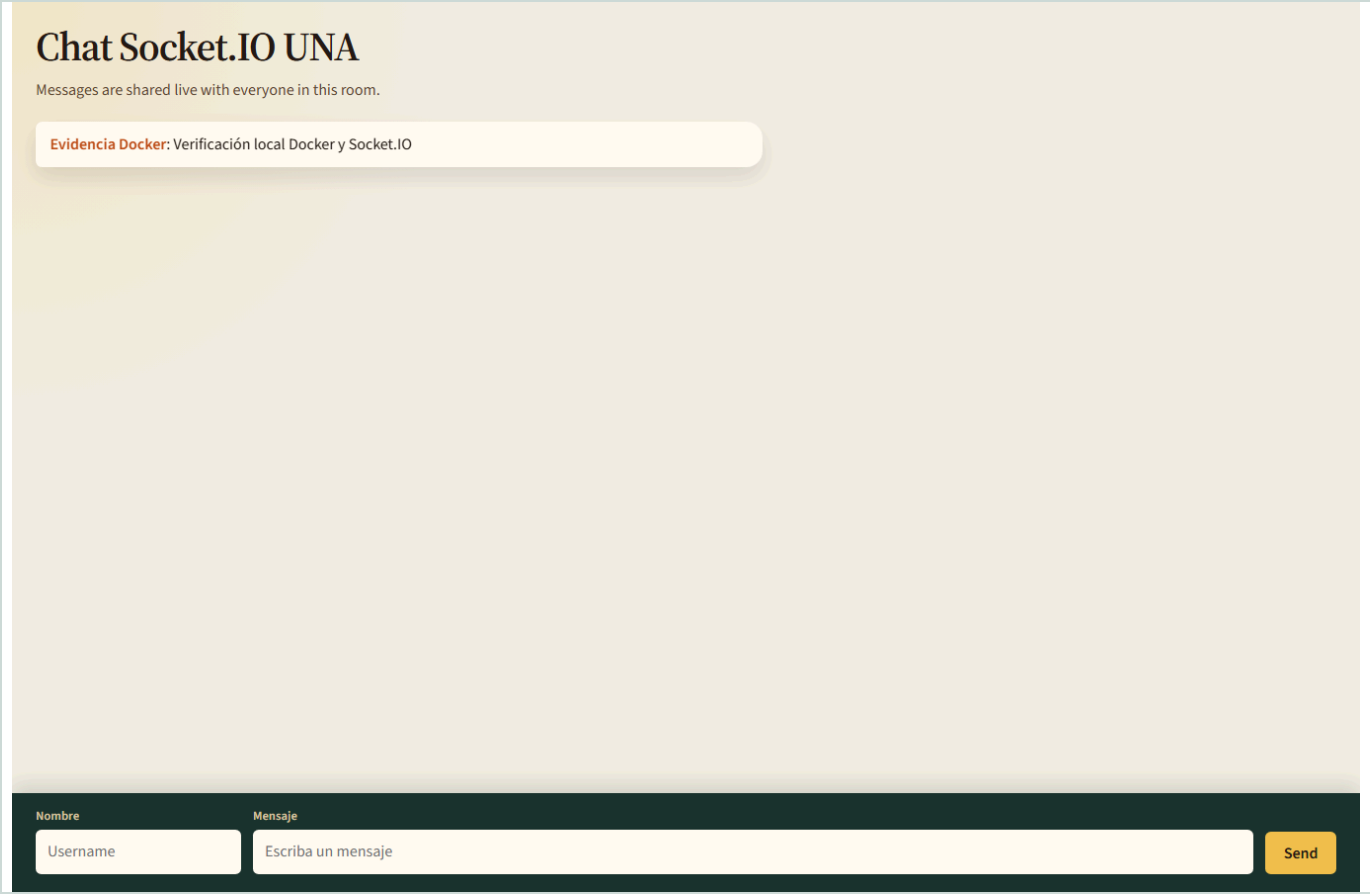
- GET `http://127.0.0.1:3000/` → HTTP 200.
- Node.js 24.x dentro del contenedor.
- Socket.IO difundió mensajes entre dos clientes locales.
- Logs Pino generaron `trace_id` y `span_id` para los mensajes; se omitió el texto del chat.

Configuración reproducible: `Dockerfile`, `docker-compose.yml` y `docker-entrypoint.sh`.

09 · PRUEBA FUNCIONAL

Dos clientes dentro de Docker

La prueba funcional conectó dos páginas al chat de localhost. Un cliente publicó el mensaje mediante el evento Socket.IO real y el otro lo recibió.



Observación de carga externa del equipo

El equipo también mantenía activo el contenedor ajeno `respondent-community-respondent-1`. Se dejó sin cambios. El uso de CPU varió durante la sesión, por lo que los percentiles deben interpretarse como medición del entorno de laboratorio, no como benchmark aislado de hardware.

Carga local: 5 usuarios por segundo

Artillery 2 usó su motor nativo `socketio` durante 60 s con `arrivalRate: 5`, un mensaje por usuario virtual. Se contaron los mensajes procesados por logs de servidor.

Métrica	Socket.IO	HTTP local
Duración / tasa	60 s / 5 VU/s	60 s / 5 VU/s
Solicitudes previstas	300 mensajes	300 POST
Procesadas / respuestas	300 procesados	300 HTTP 200
Errores	0	0
p50	3.7 ms	4.0 ms
p95	17.3 ms	8.9 ms
p99	29.1 ms	22.9 ms

Socket.IO: 300 mensajes emitidos, 300 procesados con `outcome=success`, 0 VUs fallidos. HTTP: adaptador local-only `/__lab/message`, 300 respuestas 200. No se inventaron códigos HTTP para eventos Socket.IO.

Lugar de ejecución: solo `http://127.0.0.1:3000`. Reportes en `artillery-socketio-retest-2026-10-30.txt` y `artillery-http-retest-path-2026-10-30.txt`.

11 · VALIDACIÓN Y CIERRE

Pruebas, reproducción y conclusión

Suite de aplicación

Suite (Node 24.22.2)	Resultado final
Mocha	12 passing
Jest integración	5 passing
Playwright E2E	6 passing
Total	**23/23 passing**

La auditoría de producción reportó 0 vulnerabilidades. La auditoría completa posterior a Mocha 12 quedó con 3 bajas y 24 moderadas, sin altas. En la validación final local se ejecutaron directamente las CLI con Node 24.22.2: 12 Mocha, 5 Jest y 6 Playwright aprobadas. GitHub Actions también aprobó unitarias, integración, Playwright y deploy para el commit de entrega `41b898c` (run 41180311079). En Docker se completó `npm ci --omit=dev`.

Comandos principales

```
docker compose up -d --build
npm run test:load:socketio
npm run test:load:http
curl http://127.0.0.1:3000/
```

Conclusión

El chat público y la ejecución local contenerizada funcionan; Artillery procesó los 300 mensajes previstos en localhost y Sentry recibió el evento controlado. Las variables temporales fueron retiradas, el deployment de entrega quedó Ready en Vercel y Cronitor apunta a la URL asignada. Grafana Cloud devolvió resultados consultables en Loki, Prometheus/Mimir y Tempo; el log de un mensaje Socket.IO se correlacionó con su traza mediante el mismo `trace_id`.

Anexo · índice de capturas originales

Vercel: 01-vercel-chat-funcionando; 02-vercel-dos-clientes; 09-vercel-live-chat-after-cleanup y 11-vercel-live-after-otel. GitHub Actions: 08 (run 41177108139), 12 (41179912329) y 13 (41180311079). Cronitor: 03, 04, 05 y registro vigente 10. Sentry: 06 estado inicial y 07 evento recibido. Grafana: `grafana-loki-logs`, `grafana-prometheus-requests`, `grafana-prometheus-latency`, `grafana-prometheus-errors`, `grafana-trace-chat-span-local` y `grafana-log-trace-correlated`; PNG y JPG originales con metadatos por archivo. Docker: 01-04 y registro de pruebas 05. Todo en `evidencias-laboratorio/`.

Informe y anexos: `ENTREGA-LABORATORIO.md` y `evidencias-laboratorio/README.md`.